最优化：梯度下降、牛顿法与启发式算法

读前须知

- **前置**：多元函数求偏导，知道梯度和 Hessian 分别是一阶、二阶导数的高维形式。
- **这一篇的位置**：所有模型的训练过程都是最优化问题，梯度下降则是深度学习的发动机。
- **读完能做什么**：写出梯度下降与牛顿法的更新式；判断一个目标函数是不是凸的；解释学习率过大或过小分别会发生什么。

一、把智能问题翻译成优化问题

从本质上看，人工智能的目标可以概括为**最优化**：在复杂环境与多方交互中做出最优决策。分类要最小化错误率，回归要最小化残差，训练要最小化损失——几乎所有问题最终都会落到"求某个函数的最小值"上。

最优化理论（optimization） 研究两件事：

1. 给定目标函数的最大值（或最小值）**是否存在**；
2. 如果存在，如何**找到**使目标函数取到该值的自变量取值。

一个形象的画面：把目标函数想象成一片连绵山脉，最优化就是先判断最高峰在哪里，再找一条走到山顶的路径。**注意**：机器学习语境下通常把"损失"设计成越好越小，所以习惯说"下山"而不是"上山"，两者只差一个负号。

二、目标函数、全局最优与局部最优

- 需要最小化（或最大化）的函数称为**目标函数（objective function）**，也叫评价函数、代价函数、损失函数。
- 最大化问题可以转化为最小化问题：

$$\max_x f(x) \iff \min_x \left(-f(x)\right)$$

因此讨论时统一按"求最小"处理。

2.1 全局最小与局部极小

- **全局最小值（global minimum）**：f(x*) ≤ f(x) 对定义域内所有 x 成立；
- **局部极小值（local minimum）**：f(x*) ≤ f(x) 只对 x* 的某个邻域内的 x 成立。

理想情况下当然希望找到全局最优，但那意味着必须在全局范围内搜索。继续用山脉作比：

> 想找到最高的山峰，最好的办法是站在足够高的地方，把所有山峰尽收眼底，再挑最高的一座。可惜实用的优化算法都没有这种"上帝视角"——它们只能站在山脚，一步一步往上摸索，受视野限制，找到的很可能只是方圆十里内最高的那个山头。

这就是局部最优的由来：当输入参数多、解空间大时，绝大多数实用算法都无法承受全局搜索的计算量，只能保证局部最优。

2.2 工程上的妥协

在人工智能与深度学习的实际场景里，这个限制反而没那么致命：**只要目标函数的值足够小，就可以把它当作全局最优来用。**这是一种典型的工程折中——用"近似的全局最优"换取可接受的计算复杂度。补充一点：深度网络的损失曲面通常不是简单的凸函数，存在大量局部极小与鞍点，但实践表明高维空间中"糟糕的局部极小"反而少见，这才是梯度类方法能奏效的深层原因。

> **新手提示：什么是"凸函数"**
>
> 直观上，凸函数就是一个"碗"——碗底只有一个最低点，而且从任意一点往下滚都会滚到碗底。所以对凸函数来说，**局部最优就是全局最优**，优化问题变得很简单。线性函数、二次函数（开口向上）、负熵都是凸的；而深度网络的损失函数基本都不是凸的，这也是神经网络训练难以保证最优的根源。

三、无约束优化与约束优化

按是否限制自变量的取值范围，优化问题分为两类：

- **无约束优化（unconstrained optimization）**：对 x 的取值没有额外限制；
- **约束优化（constrained optimization）**：要求 x 落在满足特定条件的集合（可行域）内。

$$\begin{aligned} \min_{x} \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \le 0,\ \ h_j(x) = 0 \end{aligned}$$

线性规划（linear programming）是最典型的约束优化：目标函数与约束都是线性的，典型场景是"有限成本下取得最大收益"。

3.1 拉格朗日乘子法

约束优化通常比无约束优化更难，但**拉格朗日乘子（Lagrange multiplier）**提供了一条通道：把含 n 个变量、k 个约束的问题，转化为含 n + k 个变量的无约束问题。

对等式约束 φ(x,y)=0，最简单的拉格朗日函数形式是：

$$L(x, y, \lambda) = f(x, y) + \lambda\, \varphi(x, y)$$

- f(x, y)：原目标函数；
- φ(x,y)：等式约束条件（φ 读作 phi）；
- λ：拉格朗日乘数，读作 lambda。

为什么可以这样转化：在满足一定正则性条件时，原问题的最优解一定是拉格朗日函数的**驻点**（梯度为 0 的点）；而且在最优点处约束成立即 φ=0，所以 L = f，两者的目标函数值相同。几何上的解释是：在最优点处，目标函数的梯度与约束曲面的梯度方向共线，λ 正是这个比例系数。**要注意**：L 本身通常没有下界，不能直接对 L 求最小值，要找的是它的驻点——这才是"转化"的准确含义。

补充说明：面对不等式约束，需要推广为 **KKT 条件（Karush-Kuhn-Tucker conditions）**，它额外要求乘子非负以及互补松弛性（$\lambda_i g_i(x) = 0$）。支持向量机的对偶推导就是 KKT 条件的经典应用；而拉格朗日对偶还能把"难解的原问题"转成"易解的对偶问题"，这是凸优化里最重要的技巧之一。

四、梯度下降：最朴素也最常用

求解无约束优化，最常用的方法是**梯度下降法（gradient descent）**。

4.1 直觉与数学依据

直觉上，梯度下降就是**沿着函数值下降最快的方向走**，像下山时专挑最陡的坡走。

> **新手提示：梯度到底是什么**
>
> 一元函数里，导数就是"斜率"，告诉你往右走一点，函数值是升还是降。多元函数有多个方向，每个方向都有一个偏导数；把所有这些偏导数按顺序排成一个向量，就是**梯度**。所以梯度可以理解为"斜率的高维版本"，它同时告诉你每个方向上的变化率。

数学上，梯度 ∇f（∇ 读作 nabla，符号本身像个倒三角）指向函数值增长最快的方向，因此下降最快的方向是**负梯度方向**。从一阶泰勒展开也能看出这一点：

$$f(x+\Delta x) \approx f(x) + \nabla f(x)^{\mathrm{T}} \Delta x$$

要让 f(x + Δx) 尽可能小，Δx 就应当取负梯度方向。

当输入是向量时，目标函数的图像变成高维曲面，梯度是**垂直于等高线、指向高度增加方向**的向量，因而携带了高维空间中的方向信息。结论可以写成一句话：**多元函数沿其负梯度方向下降最快。**这就是梯度下降的全部理论依据。

4.2 更新公式与步长

$$x_{k+1} = x_k - \eta \nabla f\left(x_k\right)$$

其中 η（希腊字母，读作 eta）是**步长（学习率，learning rate）**，也就是每次更新时 x 的变化幅度。

步长是梯度下降里最需要调的超参数：

- 太小 → 收敛慢，训练像蜗牛爬；
- 太大 → 可能一步跨过最低点，甚至在谷底两侧来回震荡，所谓"过犹不及"。

因此整体策略是**逐步减小步长**：初期大步快跑，接近最优点时小步微调。这和我们校对仪器的经验一致——先粗调，再微调。**补充说明**：常见的学习率调度包括阶梯衰减、余弦退火、预热（warmup）等；此外还有动量（momentum）方法，通过累积历史梯度抑制震荡、加速穿越平坦区域。

4.3 批处理与随机梯度下降

当训练样本很多时，梯度的计算方式有两种典型模式。

批处理（batch processing）：算出每个样本上的梯度，求和后作为本次更新的梯度。

$$\nabla f = \sum_{i=1}^{n} \nabla f_i$$

每次更新都要遍历全部训练样本，计算量大，但方向稳定。

随机梯度下降（stochastic gradient descent, SGD）：每次更新只用一个样本，下一次换另一个样本，通过不断迭代完成对所有样本的遍历。

$$x_{k+1} = x_k - \eta \nabla f_{i_k}\left(x_k\right)$$

有意思的是，当训练集规模较大时，SGD 的实际表现往往更好。原因有两层：一是单次更新代价极低，相同时间内能迭代更多次；二是单个样本带来的噪声反而有助于跳出较差的局部区域。

补充说明：工程上真正广泛使用的是折中方案 **小批量（mini-batch）**，每次取 32 ~ 1024 个样本，既保留噪声带来的探索性，又能利用矩阵运算的并行效率。今天说"用 SGD 训练"，绝大多数情况下指的是 mini-batch SGD。

五、二阶信息与牛顿法

5.1 一阶与二阶导数

梯度下降只用到**一阶导数（first-order derivative）**，也就是“函数如何随输入变化”。而**二阶导数（second-order derivative）**描述“一阶导数如何随输入变化”，提供的是**曲率（curvature）**信息。

曲率影响下降速度：

- 曲率为正（碗形向上）时，实际下降会比梯度下降预期的更慢；
- 曲率为负时，则会比预期更快。

梯度下降看不到曲率，只能利用局部一阶信息，在长长的峡谷状曲面里容易来回横跳，走出很多弯路。而不同方向上导数的长期趋势，恰恰藏在二阶信息里。

5.2 牛顿法

把二阶导数引入优化，典型方法就是**牛顿法（Newton's method）**。

做法是：先把目标函数在当前点做二阶泰勒展开（梯度下降相当于只保留一阶项），再对近似函数求导并令导数为 0。解出的方向指向这个二次近似模型的最小点，称为**牛顿方向**；它并不等同于负梯度方向，但利用了二阶曲率信息，通常更直接地指向极小点。

> **新手提示：“泰勒展开”不必惧怕**
>
> 泰勒展开就是用多项式去“局部模仿”一个复杂函数。一阶展开相当于用一条直线拟合曲面（只看斜率），二阶展开相当于用一个抛物面拟合（还看弯曲程度）。阶数越高，局部拟合越准，但计算也越贵。

$$f(x+\Delta x) \approx f(x) + \nabla f(x)^{\mathrm{T}} \Delta x + \frac{1}{2} \Delta x^{\mathrm{T}} H(x) \Delta x$$

令对 Δx 的导数为 0：

$$\nabla f(x) + H(x)\Delta x = 0 \implies \Delta x = -H(x)^{-1} \nabla f(x)$$

其中 H 是**海森矩阵（Hessian matrix）**，由二阶偏导数构成。

> **新手提示：海森矩阵是什么**
>
> 若把梯度理解为“斜率”，海森矩阵就是“斜率的变化率”，即二阶导数的多维版本。它描述曲面在每个方向上弯得有多厉害（也就是曲率）。对角线上是一个变量自己的二阶导，非对角线上是不同变量两两混合的偏导。

于是更新式为

$$x_{k+1} = x_k - H^{-1}(x_k) \nabla f(x_k)$$

相比梯度下降，牛顿法收敛更快，通常具有二次收敛速度。

代价也很大（补充说明）：海森矩阵是 n×n 的，存储开销 $O(n^2)$，求逆开销约 $O(n^3)$。当参数规模上百万甚至上亿时完全不可行。因此实践中出现了两类替代：**拟牛顿法**（如 BFGS、L-BFGS）不再直接求逆，而是用相邻两步的梯度差与位移逐步逼近海森矩阵的逆：BFGS 需要 $O(n^2)$ 的存储，L-BFGS 只保留最近 m 步的信息，把存储降到 $O(nm)$；**对角近似法**（如 Adam）只保留逐元素缩放的二阶统计量，这也是深度学习中 Adam 类优化器效果好的原因之一。

六、线性搜索与置信域：两种搜索哲学

不管是梯度下降（一阶）还是牛顿法（二阶），它们寻找最小值点的基本套路都是**先定方向、再定步长**：

$$\min_{\alpha>0} f\left(x_k+\alpha d_k\right)$$

其中 d_k 是这一轮选定的搜索方向，α（希腊字母，读作 alpha）表示沿着这个方向走多远，也就是步长。

这类算法统称为**线性搜索方法（line search）**。

还有另一类算法，顺序完全相反：**先定步长（划定区域），再定方向**。这就是**置信域方法（trust region）**。

具体流程：

1. 设定置信域半径 s，以当前点为中心、s 为半径画一个封闭球形区域；
2. 在该区域内构造目标函数的二次近似模型，求其最优点；
3. 最优点与当前点的距离就是"备选位移"；
4. 若二次近似带来**充分下降**，就移动过去，并按规则**增大** s；
5. 若下降不理想，说明步子跨太大、近似不再可信，就**缩小** s，重新计算备选位移，直到满足终止条件。

两者的差别可以这样理解：线性搜索是"先决定往哪走，再试探走多远"；置信域是"先圈定一小块可信的地盘，再在地盘内找最优点"。**补充说明**：置信域框架中最常见的子问题是"在球形区域内最小化一个二次模型"，这正是 Levenberg-Marquardt 算法处理非线性最小二乘的做法——它在高斯-牛顿法的基础上加一个阻尼项，等价于随迭代动态调整置信域半径。

七、启发式算法：向大自然借智慧

除了基于数学定理的搜索式方法，还有一类**启发式算法（heuristics）**。它的灵感来自 20 世纪 50 年代兴起的仿生学，把生物进化、物理退火等自然机理搬到优化问题上，效果不俗。

相比严谨的数学推导，启发式算法显得"返璞归真"：核心思想就是自然界"优胜劣汰"的生存法则，再在实现中加入**选择**与**突变**等经验因素。

有两句判断值得记住：

- **搜索得多，并不代表更智能**；真正的智能恰恰是善用启发式策略，不必穷举也能解决问题。
- 启发式算法**不保证最优**，但在解空间巨大、目标函数不可导时，往往是最务实的选择。

典型代表：

- **遗传算法（genetic algorithm, GA）**：模拟生物进化，通过选择、交叉、变异迭代种群；
- **模拟退火（simulated annealing, SA）**：模拟固体结晶退火过程，以一定概率接受变差的解，从而跳出局部最优；
- **蚁群算法（ant colony optimization, ACO）**：模拟蚂蚁通过信息素协作寻找最短路径的集群行为。

补充说明：这一族还有**粒子群优化（particle swarm optimization, PSO）**、**禁忌搜索（tabu search）**、**进化策略（evolution strategies, ES）** 等。需要分清的是，它们在深度学习中主要出现在**超参数搜索与神经架构搜索**这类外层任务上；而网络权重本身的训练仍然靠梯度反向传播，不属于启发式随机搜索。

四个常见误区

- **"梯度为零就是极小值"**——它只说明是驻点，也可能是极大值或鞍点。高维空间里鞍点比局部极小更常见，这也是深度学习中动量与自适应方法重要的原因之一。
- **"学习率越小越安全"**——过小会让收敛慢到不可接受，还会在平坦区域长时间停滞；过大则直接震荡发散。学习率是需要在训练中观察并调整的超参数。
- **"牛顿法总比梯度下降快"**——牛顿法收敛阶更高，但每步要算 Hessian 的逆，代价是 $O(n^3)$；非凸时还可能朝鞍点或极大值走。
- **"凸问题就一定好解"**——还要看约束是否可行、目标是否有下界。无约束的凸函数也可能没有最小值（比如线性函数）。

八、要点回顾

1. 绝大多数人工智能问题都能转化为最优化问题，通常是求目标函数的最小值。
2. 实用算法几乎只能保证局部最优；只要函数值足够小，工程上就当作全局最优使用。
3. 约束优化可通过拉格朗日乘子转化为无约束问题；不等式约束需要 KKT 条件。
4. 梯度下降沿负梯度方向更新，步长决定收敛的快慢与稳定性，总体上应逐渐变小。
5. 批处理方向稳、代价高；SGD 迭代快、带噪声；mini-batch 是二者的工程折中。
6. 牛顿法利用海森矩阵引入曲率信息，收敛更快但计算昂贵；拟牛顿法与 Adam 是常见替代。
7. 线性搜索"先方向后步长"，置信域"先步长后方向"。
8. 遗传算法、模拟退火、蚁群算法等启发式方法不保证最优，却适合不可导、超大解空间的场景。

九、名词汇总

名称	英文全称	英文缩写	简短介绍
最优化	Optimization	—	研究如何求目标函数最大值或最小值的理论与算法
目标函数	Objective Function	—	需要被最大化或最小化的函数，也称代价或损失函数
可行域	Feasible Region	—	满足所有约束条件的自变量取值范围
全局最小值	Global Minimum	—	在定义域内使函数值最小的点所对应的取值
局部极小值	Local Minimum	—	在某个邻域内使函数值最小的点所对应的取值
无约束优化	Unconstrained Optimization	—	对自变量取值不附加任何限制的优化问题
约束优化	Constrained Optimization	—	自变量必须满足若干等式或不等式约束的优化问题
线性规划	Linear Programming	LP	目标函数与约束均为线性的优化问题
拉格朗日乘子	Lagrange Multiplier	—	将约束条件并入目标函数、构造无约束问题的系数

名称	英文全称	英文缩写	简短介绍
对偶问题	Dual Problem	—	由原问题构造出的等价或近似等价的可解问题
梯度下降	Gradient Descent	GD	沿负梯度方向迭代更新参数的一阶优化算法
学习率	Learning Rate	—	每次参数更新的步长系数
动量	Momentum	—	累积历史梯度方向、抑制震荡的加速技巧
批处理	Batch Processing	—	每次更新使用全部训练样本计算梯度
随机梯度下降	Stochastic Gradient Descent	SGD	每次更新只使用单个或少量样本的无偏梯度算法
小批量	Mini-batch	—	每次更新使用一小批样本的折中训练方式
一阶导数	First-order Derivative	—	描述函数随输入变化率的导数，即梯度
二阶导数	Second-order Derivative	—	描述一阶导数变化率的导数，反映函数曲率
海森矩阵	Hessian Matrix	—	由全部二阶偏导数构成的方阵，刻画局部曲率
牛顿法	Newton's Method	—	利用二阶泰勒展开与海森矩阵求解的优化算法
拟牛顿法	Quasi-Newton Method	—	用梯度差分近似海森矩阵逆矩阵的优化方法
线性搜索	Line Search	—	先确定搜索方向、再确定步长的一类优化策略
置信域	Trust Region	—	先划定可信邻域、再在域内求最优的一类优化策略
启发式算法	Heuristics	—	借助经验规则或自然机理求解、不保证最优的方法
遗传算法	Genetic Algorithm	GA	模拟生物进化中选优、交叉与变异的全局搜索算法
模拟退火	Simulated Annealing	SA	模拟固体退火过程、以概率接受劣解跳出局部最优的算法
蚁群优化	Ant Colony Optimization	ACO	模拟蚂蚁信息素协作机制求解路径类问题的群智能算法
粒子群优化	Particle Swarm Optimization	PSO	模拟鸟群觅食、通过个体与群体最优位置更新解的算法
禁忌搜索	Tabu Search	TS	通过禁忌表避免重复访问、跳出局部最优的启发式方法